Practical No 04

Aim:
Write a Python program to demonstrate various Data Visualization Techniques.

Theory:

What is Data Visualization?
Data Visualization is the graphical representation of data and information. It allows us to present data in a visual context, such as graphs or charts, making it easier to identify patterns, trends, and correlations that might go unnoticed in raw tabular data.

In the field of data science and machine learning, visualization is a crucial step for:
- Understanding the dataset
- Detecting missing or abnormal values
- Identifying feature relationships
- Evaluating models and results

Why is Data Visualization Important?
Raw data, especially large datasets, can be difficult to interpret without proper visuals. Visualization:
- Helps understand the distribution and structure of data.
- Assists in feature selection by showing correlation and importance.
- Is used to detect outliers and anomalies in data.
- Helps in decision-making by providing clear insights.

Example:
Imagine a dataset with monthly sales of a product. A line chart can instantly show whether sales are increasing or decreasing — something that's hard to tell just by looking at numbers in a table.

Tools and Libraries Used
Python provides several powerful libraries for data visualization:

Library Description:
Matplotlib: Foundation library for static, animated, and interactive visualizations
Seaborn: Built on top of Matplotlib, provides high-level interface for attractive and informative statistical graphics
Pandas: Used for loading, manipulating, and analyzing structured data (especially CSVs)

Types of Data Visualization Techniques
Here are some commonly used techniques:

1. Line Plot
Used to visualize data over time or continuous index values.
Example: Trend of monthly sales or temperature readings.

```
plt.plot(df['Month'], df['Sales'])
```

2. Bar Chart
Used to compare values across different categories (e.g., regions, products).

```
sns.barplot(x='Month', y='Profit', data=df)
```

3. Histogram
Used to understand the distribution (spread) of a numerical column.

```
plt.hist(df['Units_Sold'])
```

4. Box Plot
Displays the spread of data through quartiles and highlights outliers.
Useful for: Comparing multiple groups.

```
sns.boxplot(x='Region', y='Sales', data=df)
```

5. Scatter Plot
Used to observe relationships (correlation) between two numeric variables.

```
sns.scatterplot(x='Sales', y='Profit', data=df)
```

6. Heatmap
A colored matrix that visualizes the correlation between features. Strong correlations are darker or brighter depending on the color palette.

```
sns.heatmap(df.corr(), annot=True)
```

Python Libraries Used in Data Visualization

1. Pandas
Library for: Data loading, cleaning, and manipulation

Overview:
Pandas stands for Python Data Analysis Library.
Provides two main data structures:
- Series (1D)
- DataFrame (2D table)

Used to: Load data from CSV, Excel, JSON, etc.

Common Uses in Visualization:
- Loading CSV files: pd.read_csv("file.csv")
- Exploring data: df.head(), df.describe(), df.columns
- Selecting columns for plotting

```
import pandas as pd
df = pd.read_csv("sales_data.csv")
print(df.head())
```

2. Matplotlib
Library for: Creating basic visualizations like line plots, bar charts, histograms, pie charts, etc.

Overview:
- One of the oldest and most widely used libraries for plotting in Python.
- Provides fine control over every element of the plot (title, labels, size, color, etc.).
- Used for both 2D and basic 3D plotting.

Common Functions:
- plt.plot() — line chart
- plt.bar() — bar chart
- plt.hist() — histogram
- plt.scatter() — scatter plot
- plt.show() — display the plot

```
import matplotlib.pyplot as plt
plt.plot(df['Month'], df['Sales'])
plt.title("Monthly Sales")
plt.xlabel("Month")
```

```
plt.ylabel("Sales")
plt.show()
```

3. Seaborn

Library for: Advanced, statistical, and attractive data visualizations

Overview:

- Built on top of Matplotlib
- Provides prettier, more informative charts with less code.
- Especially good for dataframes and statistical plots

Common Functions:

- sns.barplot() — bar chart with confidence intervals
- sns.boxplot() — shows quartiles, outliers
- sns.histplot() — distribution histogram
- sns.scatterplot() — relationship between two features
- sns.heatmap() — color-coded matrix showing correlation

```
import seaborn as sns
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
```

Why Use These Libraries Together?

Library Purpose
Pandas: Load and organize your data
Matplotlib: Create flexible and detailed plots
Seaborn: Create beautiful and statistical plots with less effort

They are often used together:
pandas prepares the data&seaborn or matplotlib visualize it.

Summary Table

Library	Full Name	Use Case
Pandas	Python Data Analysis Library	Reading, cleaning, and selecting data
matplotlib.pyplot	Plotting Library for Python	Basic plotting (line, bar, hist, scatter)
seaborn	Statistical Data Visualization	Advanced, attractive plots, heatmaps

Code:

Student Dataset (**For Batch-1 Students only**):

```
# Import libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Create a custom dataset
```

```
data = {
   'Name': ['Amit', 'Priya', 'John', 'Sara', 'Ravi', 'Neha', 'Alex', 'Kiran', 'Meena', 'Tom'],
   'Math': [85, 90, 78, 88, 76, 95, 65, 70, 92, 80],
   'Science': [80, 85, 75, 89, 72, 91, 67, 78, 90, 85],
   'English': [78, 82, 74, 85, 69, 88, 72, 76, 91, 79],
   'Gender': ['M', 'F', 'M', 'F', 'M', 'F', 'M', 'F', 'F', 'M']
}

# Convert to DataFrame
df = pd.DataFrame(data)
```

#1.Bar Chart - Math scores

```
plt.figure(figsize=(8, 5))
plt.bar(df['Name'], df['Math'], color='skyblue')
plt.title('Math Scores of Students')
plt.xlabel('Students')
plt.ylabel('Marks in Math')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

#2. Histogram - Science scores

```
plt.figure(figsize=(6, 4))
plt.hist(df['Science'], bins=5, color='lightgreen', edgecolor='black')
plt.title('Distribution of Science Marks')
plt.xlabel('Marks')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()
```

#3. Box Plot - All Subjects

```
plt.figure(figsize=(6, 5))
sns.boxplot(data=df[['Math', 'Science', 'English']])
plt.title('Box Plot of Subjects')
plt.ylabel('Marks')
plt.tight_layout()
plt.show()
```

#4. Scatter Plot - Math vs Science

```
plt.figure(figsize=(6, 5))
sns.scatterplot(x='Math', y='Science', hue='Gender', data=df, s=100)
plt.title('Math vs Science Scores')
plt.xlabel('Math')
plt.ylabel('Science')
plt.tight_layout()
plt.show()
```

#5. Line Plot - English Marks Trend

```
plt.figure(figsize=(8, 5))
plt.plot(df['Name'], df['English'], marker='o', color='purple')
```

```
plt.title('English Marks Trend')
plt.xlabel('Student')
plt.ylabel('English Marks')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

#6. Heatmap - Correlation between subjects

```
plt.figure(figsize=(6, 5))
corr = df[['Math', 'Science', 'English']].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix of Marks')
plt.tight_layout()
plt.show()
```

Iris Dataset (**For Batch-2 Students only**):

```
# Import necessary libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load sample dataset
df = sns.load_dataset('iris')  # Built-in dataset with 150 flower samples

# Set seaborn style for better aesthetics
sns.set(style="whitegrid")
```

#1. Line Plot

```
print('Line Plot')
plt.figure(figsize=(6, 4))
plt.plot(df['sepal_length'], label='Sepal Length', color='green')
plt.title('Line Plot of Sepal Length')
plt.xlabel('Sample Index')
plt.ylabel('Sepal Length (cm)')
plt.legend()
plt.show()
```

#2. Bar Plot

```
print('Bar Plot')
plt.figure(figsize=(6, 4))
sns.barplot(x='species', y='sepal_length', data=df)
plt.title('Bar Plot: Avg Sepal Length per Species')
plt.xlabel('Species')
plt.ylabel('Sepal Length (cm)')
plt.show()
```

#3. Histogram

```
print('Histogram Plot')
plt.figure(figsize=(6, 4))
```

```
plt.hist(df['petal_length'], bins=20, color='orange', edgecolor='black')
plt.title('Histogram of Petal Length')
plt.xlabel('Petal Length (cm)')
plt.ylabel('Frequency')
plt.show()
```

#4. Box Plot

```
print('Box Plot')
plt.figure(figsize=(6, 4))
sns.boxplot(x='species', y='petal_width', data=df)
plt.title('Box Plot: Petal Width per Species')
plt.xlabel('Species')
plt.ylabel('Petal Width (cm)')
plt.show()
```

#5. Scatter Plot

```
print('Scatter Plot')
plt.figure(figsize=(6, 4))
sns.scatterplot(x='sepal_length', y='sepal_width', hue='species', data=df)
plt.title('Scatter Plot: Sepal Length vs Sepal Width')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.show()
```

#6. Pair Plot

```
print('Pair Plot')
sns.pairplot(df, hue='species')
plt.suptitle('Pair Plot of Iris Dataset', y=1.02)
plt.show()
```

#7. Heatmap (Correlation Matrix)

```
print('Heatmap (Correlation Matrix)')
plt.figure(figsize=(8, 6))
# Exclude the 'species' column as it is not numeric
corr = df.drop('species', axis=1).corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Heatmap: Feature Correlation')
plt.show()
```

Result:

Hence, I have Successfully demonstrated various Data Visualization Techniques using Python program.